University of Southern Denmark

Bachelor Project

IMADA

Analysis of Real-World Developer Mistakes in configurable software systems

Student:
Salma Khair

Supervisor:
Dr. rer. nat. Sandra Greiner
Associate Professor

January 2, 2026

Contents

1 Introduction **1**
1.1 Introduction . . . 1

2 Background and State-Of-The-Art **3**
2.1 Variable Software Development . . . 3
2.1.1 Principles of Software Product Lines . . . 4
2.1.2 Classification of Edits to Configurable Software . . . 5
2.2 Principles of Natural Language Processing . . . 6
2.2.1 Basics . . . 6
2.2.2 Topic Modelling . . . 7

3 Implementation **8**
3.1 Preliminaries . . . 8
3.1.1 Supported Language Structure . . . 8
3.1.2 Tools and Libraries . . . 8
3.1.3 DiffDetective . . . 9
3.2 Architecture . . . 9
3.2.1 Overall Structure . . . 9
3.2.2 Specific Design Choices . . . 10
3.3 MyAnalysis . . . 10
3.3.1 Initialisation . . . 11
3.3.2 Classification . . . 12
3.4 Topic Analysis . . . 13
3.4.1 Input Data . . . 13
3.4.2 Keyword Frequency Analysis . . . 13
3.4.3 Output and Limitations . . . 13

4 Evaluation **15**
4.1 Setup . . . 15
4.2 Results . . . 16
4.2.1 Overview of Analysed Commits . . . 16
4.2.2 Bug-Related Commits . . . 17
4.2.3 Topic Analysis of Commit Messages . . . 18
4.2.4 Error-Related Changes . . . 18
4.2.5 Threats to Validity . . . 20
4.2.6 Discussion . . . 20

5 Conclusion **22**
5.1 Git Repository . . . 22

Chapter 1 Introduction

1.1 Introduction

Modern software systems are rarely developed as a single, fixed program. Instead, many systems are **configurable**, meaning that the same code base can be compiled into different software variants depending on hardware, platform, or selected features. A common way to implement such configurability is through conditional code guarded by **presence conditions**, for example using preprocessor directives such as `#if` and `#ifdef`. Presence conditions determine in which configurations a particular code fragment is included and therefore directly influence the behaviour of the resulting software variants.

This bachelor thesis is situated in the area of **software engineering**, with a focus on configurable software systems and variability analysis. It studies how changes to conditional code, such as presence conditions (e.g. `#ifdef`), affect software systems over time by analysing large software repositories. In standard version control systems, changes are represented as textual diffs that show added or removed lines of code. While this representation is sufficient for many development tasks, it is often insufficient for configurable systems. In such systems, even a small textual change can have a large semantic impact if it modifies a presence condition.

For example, a minor edit to a conditional expression may silently expand or restrict the set of configurations in which a code fragment is active. These effects are not visible in standard diffs and are often undocumented in commit messages. As a result, developers and tools may overlook unintended changes to variability, which can lead to configuration-dependent faults that are difficult to detect.

The problem addressed in this thesis is therefore the limited ability of traditional diffs, to explain the impact of changes to presence conditions. In particular, the thesis investigates how developer mistakes related to variability can be identified by analysing commits at the level of variability semantics, rather than raw text differences.

To address this problem, this thesis builds on the **DiffDetective** tool to explicitly identify changes in variability. DiffDetective analyses software evolution by mapping source code and annotations to structured representations, and by classifying edits according to their effect on variability. Instead of treating all changes as simple additions or deletions, DiffDetective distinguishes different kinds of variability edits, such as **refactoring** and **reconfiguration**. Based on this foundation, this thesis contributes a custom analysis tool called **MyAnalysis**. The purpose of MyAnalysis is to identify and store variability edits that are relevant for understanding developer mistakes.

The recorded results are later analysed using a topic-based approach, which groups refactoring and reconfiguration edits and collects their diff messages. This makes it easier to study variability-related diffs in more detail and to reason about recurring patterns in how such changes occur.

The analysis focuses mainly on two classes of edits: **refactoring** and **reconfiguration**. Refactor-

ing describes changes where the structure of a presence condition is modified, while its meaning remains the same. Reconfiguration describes changes where the artifact remains unchanged, but the set of configurations in which it is included changes in a non-trivial way. In addition, MyAnalysis extracts commit-level information such as commit identifiers and commit messages. A simple keyword-based heuristic is used to flag commits that are likely related to bug fixes. This allows the analysis to connect variability edits with developer intent as expressed in commit messages.

The solution is evaluated by applying MyAnalysis to a real-world configurable software system. By analysing how often variability-related diffs occur in practice, examining which types of diffs are most common, and studying the distribution of different variability edit types across large software repositories, the evaluation makes it possible to assess the overall extent of the problem.

This approach allows us to determine, whether changes to presence conditions occur frequently and whether they represent a more substantial issue than initially expected. The results show that the analysis is able to distinguish, between different types of variability changes and provides a more transparent view of how presence conditions evolve over time. By going beyond traditional textual diffs, the approach makes variability changes visible that would otherwise be difficult to recognise in standard version control output.

In the following chapters, this thesis is structured as follows. The next chapter provides background on configurable software and presence conditions and explains the motivation for using variability-aware differencing. The subsequent chapter describes the design and implementation of MyAnalysis. Furthermore, the evaluation chapter presents the study and discusses the results. Finally, the thesis concludes with a summary of the main findings and an outline of possible directions for future work.

Chapter 2 Background and State-Of-The-Art

This chapter presents the theoretical background and state of the art, relevant to this bachelor thesis. It introduces the fundamental concepts of **configurable software systems**, and explains how variability is implemented using **presence conditions** and conditional compilation. Furthermore, it presents existing approaches for reasoning about edits to variable source code, including the classification of such edits. Finally, the chapter introduces basic principles of **Natural Language Processing (NLP)** that are later used to analyse commit messages. Together, these topics establish the conceptual basis for analysing developer mistakes related to presence conditions in configurable software systems.

2.1 Variable Software Development

Variable or configurable software systems are explicitly designed to support multiple variants within a single shared code base. As described by Apel et al., "a software product line is a set of software-intensive systems that share a common, managed set of features satisfying the specific needs of a particular market segment or mission and that are developed from a common set of core assets in a prescribed way" [1]. By selecting different configuration options or features, distinct software variants can be derived from the same underlying implementation. Each configuration determines which parts of the system are active and which are excluded, resulting in a concrete software product tailored to a specific context while maximising reuse of shared assets [1].

Such configurable software systems are widely used in practice, particularly in large and long-lived systems where maintaining separate forks would be infeasible. Operating systems are a prominent example: an operating system kernel must support different hardware architectures, device drivers, and optional subsystems, all of which are not needed in every deployment. Similarly, system libraries often provide optional functionality that can be enabled or disabled depending on performance requirements or supported platforms. Server applications frequently rely on configurability to support different protocols, security mechanisms, or deployment environments, while embedded systems must adapt to strict hardware constraints and diverse device capabilities. As noted by Apel et al., "variability is essential to adapt software systems to different execution environments, platforms, and requirements" [1]. Across these domains, configurability enables reuse of common functionality while still allowing systems to be customised to specific needs.

In real-world systems, variability is commonly implemented using directive-based mechanisms that conditionally include or exclude code. In languages such as C and C++, this is typically realised through conditional compilation using preprocessor directives such as `#if`, `#ifdef`, and `#ifndef`. These directives guard code fragments with logical expressions over configuration options. As observed by Liebig et al., "the C preprocessor is widely used to implement variability by conditionally including or excluding code fragments" [3]. Such expressions are referred to as presence conditions, as they specify under which configurations a particular code fragment is present in the compiled program. Given a concrete selection of configuration options, all code whose presence conditions evaluate to false is excluded before compilation, resulting in a customised software variant [1].

While conditional compilation allows for variability at the level of individual statements or blocks of code, it also introduces substantial complexity. Presence conditions are often hierarchical and combined across multiple levels of the source code, leading to complex logical expressions that are not explicitly represented in a single location. Apel et al. emphasise that "conditional compilation easily leads to scattered and tangled variability, which complicates understanding and maintenance" [1]. As a consequence, the effective presence condition of a code fragment depends not only on its local annotation but also on all enclosing conditions. Liebig et al. similarly report that "understanding the impact of a change requires reasoning about the complete nesting structure of preprocessor directives" [3]. Understanding which configurations are affected by a given piece of code therefore requires reasoning about the complete conditional structure rather than an individual directive.

This complexity significantly affects how developers reason about variable software. Instead of dealing with a single program, developers must reason about a potentially enormous set of variants. In large systems, the number of valid configurations can grow into thousands or even millions, making exhaustive testing or manual inspection infeasible. As a result, developers typically rely on approximation strategies, such as sample-based testing or family-based analysis rather than analysing all variants explicitly. As discussed by Apel et al., "due to the exponential number of variants, developers usually resort to sampling techniques or family-based analyses instead of analysing each variant individually" [1].

Despite the widespread use of configurable software systems, computer scientists and practitioners still face challenges in fully understanding and managing variability. Traditional software engineering tools, such as version control systems and diff tools, operate primarily on plain text and are largely unaware of the semantics of presence conditions. As noted by Apel et al., "existing development tools are mostly oblivious to variability and do not make the effects of variability explicit" [1]. As a result, changes to variability, especially changes to presence conditions, may not be immediately visible during code review. A seemingly small textual edit can unintentionally alter the set of configurations in which code is included, without this effect being explicitly documented or easily detectable.

This lack of tool support for configurable software, increases the risk of developer mistakes when editing such systems. In particular, edits to presence conditions may introduce unintended changes to variability, potentially leading to configuration-dependent bugs that are difficult to detect and reproduce. Liebig et al. emphasise that "variability-related changes are hard to understand because their effects are often implicit and scattered across the code base" [3]. Understanding how variable software systems are implemented in practice, and how developers interact with presence conditions during evolution, is therefore a crucial prerequisite for analysing real-world developer mistakes, in configurable software systems.

2.1.1 Principles of Software Product Lines

To reason systematically about variability in configurable software systems, the concept of a software product line provides an important foundation. Rather than focusing on individual variants in isolation, a software product line captures the idea that multiple related software products can be derived from a shared set of assets, by varying certain characteristics in a controlled way. As defined by Apel et al., "a software product line is a set of software-intensive systems that share a common, managed set of features and are developed from a common set of core assets" [1].

A central concept in this context is that of a *feature*. A feature represents a user-visible characteristic or capability of a software system. Features describe functionality at a level that is meaningful to users or stakeholders, such as support for a specific platform, optional functionality, or performance-related behaviour. According to Apel et al., "features are increments in program functionality that are relevant to some stakeholder and are used to distinguish between different

products of a product line" [1]. Features can be mandatory, meaning that they are present in all products of the product line, or optional, meaning that they are only included in some products. In more complex cases, features may be organised into alternative or grouped structures, where only specific combinations are allowed.

A configuration is a concrete selection of features that defines a specific variant of the software system. Not all combinations of features are valid, as constraints may restrict how features can be combined. These constraints are commonly captured using feature models, which specify which feature combinations are allowed and which are forbidden. Apel et al. describe feature models as "a compact representation of all products of a product line in terms of features and their relationships" [1]. Each valid configuration corresponds to a concrete software product, that is, an individual variant of the system that includes exactly the functionality specified by the selected features.

While a configuration describes a single software product, a software product line encompasses the entire family of related products that can be derived from different valid configurations. All products in the product line share common functionality but differ with respect to selected features. As emphasised by Apel et al., "software product lines aim at systematic reuse by managing commonality and variability in a disciplined way" [1].

2.1.2 Classification of Edits to Configurable Software

Edits to configurable software do not only affect the implementation of source code but may also change the variability information that determines in which software variants the code is included. In systems that rely on presence conditions, even small edits can significantly alter the set of configurations in which a code fragment is active. Understanding such changes therefore requires reasoning about the semantic impact of edits on presence conditions, rather than only their textual representation. As noted by Bittner et al., "small syntactic changes can have a large semantic impact on the variability of a system" [2].

To address this challenge, Bittner et al. (2022) propose the first complete and unambiguous classification of edits to variability in source code. Their classification is based on a formal model of variability and explicitly captures how edits affect feature mappings and presence conditions over time. According to Bittner et al., "we present a complete and unambiguous classification of variability edits that precisely describes how presence conditions evolve" [2]. This classification serves as the conceptual foundation for the edit analysis conducted in this thesis.

Figure 2.1 provides an overview of the edit classification proposed by Bittner et al. (2022). The figure illustrates how variability-related edits are structured into distinct categories, based on their semantic effect on presence conditions and feature mappings.

At a high level, edits are categorised according to whether they add, remove, or modify variability. The classification distinguishes edits, that introduce or delete code together with variability information, from those that change the presence conditions of existing code without modifying the code itself.

The classification defines eight fundamental edit categories. Additive edits include **AddWithMapping**, which describes the addition of new code together with a new presence condition, and **AddToPC**, which captures additions of code that reuse an existing presence condition. Subtractive edits include **RemWithMapping** and **RemFromPC**, which describe different forms of code removal with respect to variability annotations.

More subtle edits affect the variability of unchanged code. A **Generalization** makes a presence condition less restrictive, increasing the number of configurations in which code is included, while

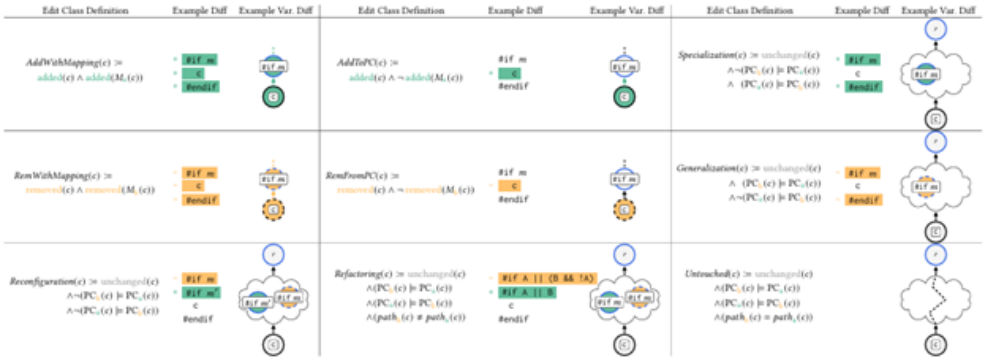

Figure 2.1: Classification of edits to configurable software proposed by Bittner et al. (2022).

a **Specialization** makes a presence condition more restrictive. A **Reconfiguration** replaces a presence condition with a non-comparable alternative, leading to a non-monotonic change in the configuration space. Finally, **Refactoring** changes the syntactic structure of a presence condition while preserving its semantics.

A key contribution of this classification is that it is complete and unambiguous: every variability related edit can be assigned to exactly one category. As stated by Bittner et al., "each variability edit belongs to exactly one edit class, making the classification complete and mutually exclusive" [2]. Traditional line-based diff tools cannot capture these distinctions, as they ignore the semantics of presence conditions. The classification therefore provides a necessary abstraction for tools such as **DiffDetective** and for the analysis performed in this thesis.

2.2 Principles of Natural Language Processing

2.2.1 Basics

Natural Language Processing (NLP) is a subfield of computer science and artificial intelligence that focuses on the automatic analysis of human language. The central idea behind NLP is to transform unstructured text into representations that can be processed, analysed, and compared computationally. In software engineering research, NLP is commonly applied to textual artefacts such as commit messages, issue reports, and documentation.

While tools such as DiffDetective make variability-related edits explicit at the code level, commit messages remain a primary source for understanding why such edits were performed. NLP provides a systematic way to analyse these messages across large commit histories.

Despite its usefulness, NLP has inherent limitations, as it can only operate on the information explicitly provided in the text. Commit messages are intended to provide a complete summary of a change, but in practice they often cannot capture every technical detail, particularly when a single commit bundles multiple modifications. While commit messages are frequently informal in style, they may still omit information that is relevant for understanding the concrete impact of a change on the code base. Consequently, NLP techniques cannot compensate for missing or unreported details and should therefore be used as a complementary technique alongside code-based analyses.

2.2.2 Topic Modelling

While DiffDetective identifies how presence conditions change at the source-code level, topic modelling is used to analyse commit messages in order to understand the contexts and intentions under which these changes were made. Topic modelling is an unsupervised NLP technique that discovers underlying thematic structures in a collection of documents.

One of the most widely used topic modelling techniques is **Latent Dirichlet Allocation (LDA)**, a probabilistic generative model that represents documents as mixtures of topics. In this thesis, topic modelling is applied to commit messages associated with edits to presence conditions identified by **DiffDetective** and `MyAnalysis`. Each commit message is treated as a document, and the resulting topics capture recurring themes in developer communication related to variability changes, such as bug fixing, cleanup of conditional compilation directives or refactoring and reconfiguration edits to presence conditions.

@articleblei2003lda, author = David M. Blei and Andrew Y. Ng and Michael I. Jordan, title = Latent Dirichlet Allocation, journal = Journal of Machine Learning Research, volume = 3, year = 2003, pages = 993–1022

Chapter 3 Implementation

This chapter describes the implementation of the analysis conducted in this bachelor thesis. It introduces the technical prerequisites of the implementation, outlines the supported language structures and tools, and presents the architecture of the analysis built on top of DiffDetective. The chapter further explains the specific design choices made for the custom analysis and discusses how the correctness of the implementation is ensured.

3.1 Preliminaries

This section describes the technical scope of the implementation and the tools used to realise the analysis.

3.1.1 Supported Language Structure

The implementation focuses on analysing configurable C/C++ software systems that use conditional compilation to express variability. In particular, the analysis supports variability implemented using preprocessor directives such as `#if`, `#ifdef`, `#ifndef`, `#else`, and `#endif`. These directives are used to guard code fragments with presence conditions that determine under which configurations the code is included.

The analysis does not operate on fully pre-processed code. Instead, it relies on annotated source code in which conditional compilation directives are still present. This focus is essential, as the analysis targets edits to presence conditions rather than fully configured variants.

Other variability mechanisms, including runtime configuration options and build-system-level variability, are outside the scope of this implementation. The analysis therefore concentrates on preprocessor-based variability, which is widely used in real-world C/C++ systems and well supported by existing variability analysis techniques.

3.1.2 Tools and Libraries

The implementation is realised using a combination of existing analysis frameworks and standard development tools. The analysis is implemented in Java and is designed to process real-world software repositories with publicly available version control histories.

Git is used to access and traverse commit histories, enabling the analysis of software evolution at the level of individual commits. In particular, the implementation uses the JGit Java library to analyse public GitHub repositories by programmatically cloning repositories and traversing their

commit histories. Standard Java libraries are used for file handling, data processing and result serialisation, supporting the structured storage of analysis results for further evaluation.

In addition, the implementation integrates a variability-aware differencing framework, DiffDetective, that provides the necessary abstractions for analysing configurable software systems. DiffDetective enables the analysis to reason about presence conditions and their evolution across commits. The specifics of this framework and its role in the implementation are described in the following section.

3.1.3 DiffDetective

DiffDetective is a variability-aware differencing Java library designed to analyse the evolution of configurable software systems. Unlike traditional line-based diff tools, DiffDetective explicitly models variability by representing conditional compilation directives and presence conditions as structured entities rather than plain text.

DiffDetective parses annotated source code and represents the association between code fragments and their corresponding presence conditions. When comparing two revisions of a source file, the framework detects differences in these associations, allowing variability-related edits to be identified even when the underlying code remains unchanged.

A central feature of DiffDetective is its ability to classify edits according to their semantic impact on variability. Based on the formal edit classification proposed by Bittner et al. [2], DiffDetective distinguishes between different classes of variability-related edits, including refactoring, reconfiguration, generalisation, and specialisation. This classification provides a precise vocabulary for describing how presence conditions evolve across commits.

In this thesis, DiffDetective serves as the foundation for identifying and classifying edits to presence conditions. The framework provides the core analysis capabilities, while the custom analysis component builds on its results to extract relevant metadata and prepare the data for further evaluation and topic-based analysis of commit messages.

3.2 Architecture

3.2.1 Overall Structure

The execution of the analysis starts in the Java component and proceeds in a sequential manner. The entry point of the implementation is the class `Main.java`, which is responsible for configuring the analysis and connecting it to DiffDetective's execution infrastructure. `Main.java` defines the repository location and output directory and executes DiffDetective's traversal over individual commits. Importantly, this class does not implement analysis logic itself; instead, it delegates execution to DiffDetective, which ensures that the repository history is traversed consistently and reproducibly.

```
Path repoPath = Paths.get("repositories/postgresql");
Path outputDir = Paths.get("output");

Repository repo = Repository.fromDirectory(repoPath,
    repoPath.getFileName().toString());
Analysis.forEachCommit(() -> MyAnalysis.Create(repo, outputDir));
```

Listing 3.1: Entry point and configuration in `Main.java`

For each pair of successive commits, DiffDetective computes a variability-aware diff that captures changes to annotated source code, including the association between code fragments and their presence conditions. These diffs are represented internally using structured models rather than plain text. The custom analysis, `MyAnalysis`, is executed within this traversal and receives access to both the current commit and the corresponding variability-aware diff.

Architecturally, this design cleanly separates control flow from analysis. Repository traversal, parsing of annotated source code, and computation of variability-aware diffs are handled entirely by DiffDetective. The custom implementation only observes and processes the results of these computations. This separation reduces complexity and ensures that the custom analysis remains focused on the specific goals of the bachelor thesis.

Lastly, a topic analysis is applied to structure the extracted results. This analysis groups related observations and supports the identification of recurring patterns in the evolution of variability across commits.

3.2.2 Specific Design Choices

A key characteristic of the implementation is that it builds a custom, commit-level analysis on top of an existing variability-aware differencing infrastructure, rather than reimplementing repository traversal or differencing logic. By leveraging DiffDetective for commit traversal, parsing of annotated source code, and computation of variability-aware diffs, the implementation focuses exclusively on analysing and interpreting the resulting data.

Another specific aspect is that the analysis operates directly on real-world repositories with publicly available version control histories, without requiring preprocessing or manual annotations beyond those already present in the code base. The analysis is executed sequentially per commit, which enables fine-grained reasoning about how variability and presence conditions evolve over time.

Finally, the integration of a topic analysis step distinguishes the implementation by supporting higher-level interpretation of the extracted results. Rather than producing only raw diffs, the implementation structures observations into recurring themes, which makes it easier to evaluate how variability evolves over time.

3.3 MyAnalysis

MyAnalysis constitutes the custom analysis component of the implementation and is responsible for interpreting the variability-aware diffs computed by DiffDetective. Rather than performing repository traversal or syntactic differencing itself, MyAnalysis is integrated into DiffDetective's execution pipeline and operates exclusively on the intermediate results produced during commit traversal.

The analysis is implemented by conforming to the `Analysis.Hooks` interface provided by DiffDetective. This interface defines a set of lifecycle methods that are invoked at specific points during commit traversal. An instance of MyAnalysis is registered through a factory method, which

constructs an `Analysis` object and attaches MyAnalysis as an analysis hook. Once registered, MyAnalysis is executed automatically for each commit processed by DiffDetective.

During execution, DiffDetective iterates over the commit history of the analysed repository. For each commit, it computes a variability-aware diff that captures changes to annotated source code together with their associated presence conditions. MyAnalysis accesses both the current commit object and the corresponding diff through the analysis context, allowing it to analyse commit metadata and variability-related changes in a unified manner.

3.3.1 Initialisation

```
@Override
public void initializeResults(Analysis analysis) {
    // Ensure that the output directory exists
    Files.createDirectories(outputDirectory);

    // Prepare analysis log file
    Path analysisFile = Paths.get(
        System.getProperty("user.dir"),
        "commit_analysis_log.txt"
    );
    Files.deleteIfExists(analysisFile);
    Files.createFile(analysisFile);

    // Writer for structured per-commit analysis results
    analysisWriter = new BufferedWriter(
        new FileWriter(analysisFile.toFile(), true)
    );

    ...

    // Reset internal state to avoid duplicate output
    writtenCommits.clear();

    ...
}
```

Listing 3.2: Initialisation of analysis output in `MyAnalysis`

The initialisation of the analysis is performed in the `initializeResults` method of the `MyAnalysis` class, as illustrated in Listing 3.2. This method prepares the output environment before commit traversal begins by ensuring that the output directory exists and by resetting previously generated result files to guarantee reproducibility across multiple executions.

During initialisation, two output files are created: `commit_analysis_log.txt` and `commit_messages.txt`. The former stores a compact, structured summary of the analysis results for each commit, including commit identifiers and classification information. The latter stores the raw commit messages associated with each analysed commit. Separating these outputs allows structured data to be processed automatically while preserving commit messages for later manual inspection or qualitative analysis.

To prevent duplicate output, the `MyAnalysis` class maintains a set of commit identifiers that have already been processed. Before writing results for a commit, the analysis checks whether the corresponding identifier has already been recorded.

3.3.2 Classification

The classification of variability-related changes is implemented in the `classifyDiff` method of the `MyAnalysis` class. This method operates on the variability-aware diff computed by DiffDetective and iterates over the artifact nodes contained in the diff structure.

```
private EditClass classifyDiff(VariationDiff<?> diff) {
    // Stores the last applicable edit class encountered
    EditClass editClass = null;

    try {
        // Iterate over all artefact nodes in the variability-aware diff
        for (org.variantsync.diffdetective.variation.diff.DiffNode<?>
            node :
                diff.computeAllNodesThat(n -> n.isArtifact())) {

            // Match the current node against DiffDetective's proposed
                edit classes
            editClass = ProposedEditClasses.Instance.match(node);
            if (editClass == null) continue;

            // Prioritise higher-level variability edits
            if (editClass.equals(ProposedEditClasses.Refactoring)
                    ||
                        editClass.equals(ProposedEditClasses.Reconfiguration))
                        {
                return editClass;
            }
        }
    } catch (Exception e) {
        // Log classification errors without interrupting the analysis
        Logger.error("Error classifying diff: {}", e.getMessage());
    }

    // Return the last matched edit class if no prioritised class was
        found
    return editClass;
}
```

Listing 3.3: Classification of variability-aware edits in `MyAnalysis`

For each artifact node, the method applies DiffDetective's proposed edit-class matching mechanism to assign a semantic edit class. The implementation prioritises higher-level edit classes such as refactorings and reconfigurations. If such a classification is detected, the method terminates early and returns the corresponding edit class. If no prioritised edit class is found, the method returns `null`, indicating that no relevant edit class was detected by this heuristic.

In addition to structural change classification, MyAnalysis analyses commit messages to identify commits that are likely related to bug fixing. This logic is implemented using a lightweight keyword-based heuristic.

```
private boolean isBugRelated(String message) {
    if (message == null) return false;
    String lower = message.toLowerCase();
    return lower.contains("bug")
        || lower.contains("fix")
        || lower.contains("error")
        || lower.contains("issue")
        || lower.contains("defect");
```

```
}
```

Listing 3.4: Heuristic for detecting bug-related commits

A commit is classified as bug-related if its message contains commonly used terms associated with bug fixes, such as *bug*, *fix*, *error*, *issue*, or *defect*. While this heuristic does not guarantee perfect accuracy, it enables efficient labelling of commits and supports later grouping and interpretation of results.

3.4 Topic Analysis

The topic analysis is implemented as a lightweight post-processing step that operates on the output produced by MyAnalysis. Its purpose is to structure the commit-level results into higher-level themes by analysing the vocabulary used in commit messages for selected categories of commits.

3.4.1 Input Data

The topic analysis consumes the files `commit_analysis_log.txt` and `commit_messages.txt`. The first file contains one structured summary line per commit, including the commit identifier and assigned edit class. The second file contains the raw commit messages.

Commits classified as *Refactoring* or *Reconfiguration* are selected by parsing `commit_analysis_log.txt`. Only the commit messages corresponding to these identifiers are included in the subsequent analysis.

```
refac_types = {"Refactoring", "Reconfiguration"}
if ctype in refac_types:
    interesting_ids.add(cid)
```

Listing 3.5: Selecting refactoring and reconfiguration commits from the analysis log

3.4.2 Keyword Frequency Analysis

To approximate topics, a keyword frequency analysis is performed over the selected commit messages. Messages are tokenised into lowercase words using a regular-expression-based tokenizer. Common stopwords and noise terms are removed, and very short tokens are excluded. The remaining words are counted to produce a ranked list of frequently occurring keywords.

```
text = re.sub(r'[^a-zA-Z\s]', '', text).lower()
words = [w for w in text.split() if len(w) > 2]
counter = Counter(words)
most_common = counter.most_common(10)
```

Listing 3.6: Keyword frequency analysis over selected commit messages

3.4.3 Output and Limitations

The results of the topic analysis are written to a human-readable report file. The report includes the number of analysed commits, the most frequent keywords, and the commit messages used for

the analysis. This supports both quantitative inspection and qualitative validation.

The topic analysis is intentionally lightweight and does not claim to perform semantic topic modelling. Instead, it provides a transparent, frequency-based approximation of recurring themes. The results depend on the quality and consistency of commit messages and may reflect repository-specific terminology. Despite these limitations, the approach is efficient and suitable for supporting systematic interpretation of large commit histories.

Chapter 4 Evaluation

This chapter evaluates the implementation presented in this thesis. It describes the experimental setup, reports the results obtained from analysing the selected subject systems, and discusses threats to the validity of the evaluation. In addition, the findings are reflected upon in a subsequent discussion, where the results are interpreted, limitations are considered, and the strengths and weaknesses of the proposed approach are examined.

4.1 Setup

The evaluation is conducted on a set of real-world, configurable software systems that use preprocessor-based variability. These systems constitute the subject systems of the study. The selection of subject systems was performed incrementally. Initially, smaller repositories were chosen in order to validate the analysis workflow and to assess the behaviour of the implementation on a manageable commit history. As an initial subject system, the repository `xterm` was selected. The analysed revision range of `xterm` contains 112 commits and exhibits non-trivial use of conditional compilation. This limited size made it possible to inspect the analysis results in detail and to validate the correctness of the implementation before scaling to larger systems. After successfully analysing `xterm`, additional subject systems with larger commit histories were considered. In particular, the repository `libssh` was analysed, which contains a substantially larger number of commits and more complex variability-related code. This allowed the evaluation to assess the behaviour of the implementation on a more realistic and demanding subject system.

Each subject system was cloned locally and analysed using `DiffDetective` in combination with the custom analysis component `MyAnalysis`. The repositories were provided to DiffDetective as input and processed commit by commit. For each pair of successive commits, DiffDetective computed a variability-aware diff, capturing changes to annotated source code together with their associated presence conditions. `MyAnalysis` was executed as part of this traversal and recorded commit-level metadata and classification results.

The evaluation focuses on measuring variability-related changes at commit level. For each analysed commit, the implementation records the assigned variability edit class, as provided by DiffDetective, and whether the commit message indicates a potential bug fix based on a keyword-based heuristic. These measurements support both quantitative analysis, such as counting the frequency of different edit classes, and qualitative analysis, such as inspecting individual commit messages and corresponding code changes [**selfAdmittedIssues2023**].

To support higher-level interpretation of the results, a topic analysis was applied as a post-processing step. The topic analysis focuses specifically on commits classified as *Refactoring* or *Reconfiguration*, as these represent higher-level changes to variability. A lightweight keyword-frequency approach was used to analyse the commit messages associated with these commits, allowing recurring terms and themes to be identified without relying on complex semantic models [**selfAdmittedIssues2023**].

4.2 Results

This section presents the results of applying the proposed analysis to the selected subject systems. The results are reported both quantitatively and qualitatively, focusing on (1) how often different types of variability-related changes occur, (2) how frequently commits are classified as bug-related, and (3) which recurring themes can be observed through topic analysis of commit messages. In addition, selected commits are inspected manually to validate and interpret the automated results and to illustrate how variability-related changes manifest in practice.

4.2.1 Overview of Analysed Commits

Across all subject systems, the analysis processes each repository commit by commit using DiffDetective together with the custom analysis component `MyAnalysis`. For each commit, a variability-aware diff is computed and classified, and the corresponding commit message is analysed. This enables a combined view of structural changes to variability and the developers' textual explanations of those changes.

As an initial step, smaller repositories were analysed to validate the feasibility and correctness of the approach. The first subject system was `xterm`, which contains approximately 112 commits in the analysed revision range. This repository was chosen deliberately because of its limited size, which made it possible to manually inspect results and correlate automated classifications with actual code changes in detail.

After validating the approach on `xterm`, larger repositories such as `libssh` were analysed. These systems contain substantially more commits and more complex variability, allowing the evaluation to assess how the approach scales and how well it performs on realistic, non-trivial software systems.

DiffDetective traverses the complete commit history of each subject system (for example, 5 352 commits for `libssh`). However, the evaluation dataset reported in Table 4.1 is based on the commits that were successfully recorded in the per-commit output logs (`commit_analysis_log.txt` and `commit_messages.txt`). The analysis is executed with multi-threading enabled, and the output files are reset during initialisation. As a result, the final log files may contain only a subset of the traversed commits. Consequently, the number of commits reported in Table 4.1 should be interpreted as the number of commits available for evaluation in the produced logs, rather than the total number of commits in the repository history.

For `xterm`, 111 commits were analysed, none of which were classified as bug-related based on commit-message analysis.

For `libssh`, DiffDetective traversed the full commit history of 5 352 commits. However, due to repeated initialisation of the analysis and overwriting of output files during multi-threaded execution, the final per-commit log contains only a subset of the traversed commits. Consequently, quantitative results for `libssh` are interpreted qualitatively rather than as absolute frequencies.

Despite this limitation, the analysed subset of `libssh` commits shows a clear tendency toward corrective maintenance. A substantial number of commits in the final log explicitly indicate corrective intent in their messages (e.g. using terms such as *fix*, *bug*, or *error*). This observation suggests that developers in `libssh` frequently document errors and fixes in commit messages, particularly when modifying conditional compilation logic, and stands in contrast to the `xterm` repository, where such intent is largely absent from commit messages.

xterm commit messages

```
Commit ID: 73ac00b3
xterm-331

Commit ID: 5c791e88
xterm-330
```

libssh commit messages

```
Commit ID: 41dd2a2a
Check return values of option setter.

Commit ID: 2aa81c05
Add memory error checks for strdup().
```

Figure 4.1: Comparison of commit-message styles in xterm and libssh. xterm commits use short, identifier-based messages with little semantic information, whereas libssh commits explicitly document corrective intent and error-related changes.

```
Commit: 7ceaf5c43af77666093638e7a5e67faf86aba9cc
Type: Reconfiguration
Bug-related: true
Message: Fix the ifdefs.
git-svn-id: svn+ssh://svn.berlios.de/svnroot/repos/libssh/trunk@235
```

Figure 4.2: Example of a bug-related reconfiguration commit in `libssh`. The commit message explicitly indicates corrective intent, and the edit class shows that an existing presence condition was modified rather than new functionality being added.

4.2.2 Bug-Related Commits

In addition to the structural classification of variability-related changes, the analysis identifies commits that are likely related to bug fixing based on commit-message keywords. This approach follows prior work on self-admitted issues, which demonstrates that commit messages often encode developers' corrective intent and can therefore be used to study maintenance activities.

The analysed subject systems differ strongly with respect to bug-related commits. In xterm, none of the analysed commits were classified as bug-related based on commit-message analysis. In contrast, the analysed subset of commits from libssh shows a clear tendency toward corrective maintenance, which can be seen in the figure Figure 4.2. A substantial number of commit messages explicitly contain terms such as *fix*, *bug*, *error*, *leak*, or *segfault*. This observation is consistent with findings from self-admitted issues research, which report that developers frequently document defects and fixes directly in commit messages using natural language.

The high occurrence of bug-related commits in `libssh` suggests that variability-related code is often involved in corrective maintenance. This is particularly evident in low-level system components that rely heavily on conditional compilation and configuration-dependent behaviour. In such contexts, incorrect or incomplete presence conditions can easily lead to build failures, runtime errors, or missing functionality, which developers subsequently address by reconfiguring or refactoring preprocessor directives.

Manual inspection of selected bug-related commits confirms that the automated classification aligns with the actual changes in most cases. Typical examples include fixing incorrect `#ifdef` conditions, adding missing error checks under specific configurations, or correcting platform-dependent compilation paths. At the same time, the analysis also reveals limitations of the keyword-based heuristic. Some commits are classified as bug-related even though they primarily address maintainability concerns, such as fixing build warnings or cleaning up configuration logic. These cases illustrate that commit-message-based classification captures corrective intent rather than definitive runtime faults and should be interpreted accordingly.

4.2.3 Topic Analysis of Commit Messages

To further interpret variability-related changes, the commit messages associated with refactoring and reconfiguration edits were analysed to identify recurring themes. The results reveal a small and stable set of keywords that occur repeatedly across variability-related commits, providing insight into the motivations behind variability evolution.

Frequently occurring terms include *fix*, *ifdef*, *add*, *support*and configuration-related identifiers. The repeated presence of these keywords indicates that changes to variability are predominantly driven by corrective maintenance and configuration management rather than by the introduction of new functionality. In particular, references to conditional compilation suggest that developers frequently revise and correct presence conditions to resolve misconfigurations or build-related issues. Results from the topic analysis, keyword frequencies, is illustrated in Figure 4.3.'

The recurring nature of these terms also suggests that the semantic impact of changes to conditional compilation directives is difficult to anticipate. Although edits to `#if`- or `#ifdef`-based logic often appear local, they can alter the set of configurations in which code is included, leading to unintended effects that only become visible later. Overall, the topic analysis supports the conclusion that variability evolution in real-world systems is largely driven by ongoing maintenance and error correction, highlighting variability management as a recurring and non-trivial challenge during software evolution.

4.2.4 Error-Related Changes

In addition to classifying variability-related changes, the evaluation analyses how often commits are associated with errors and how such errors manifest in practice. Error-related commits are identified using a lightweight, keyword-based heuristic applied to commit messages, as described in Chapter 3. A commit is considered error-related if its message contains commonly used terms such as *bug*, *fix*, *error*, *issue*, or *defect*. Following prior work on self-admitted issues, this heuristic is interpreted as an indicator of corrective intent rather than a definitive classification of functional defects.

In addition to error-related commits inferred from commit messages, the analysis also reports errors that occur during the parsing of annotated source code and presence conditions. These parsing errors indicate situations in which real-world preprocessor usage violates assumptions of a well-formed annotation structure. For `libssh`, the exported metadata reports 97 occurrences of line-continuation errors, 34 cases of unclosed annotations, 4 occurrences of `#else` or `#elif` without a matching `#if`, and 6 cases where the extracted presence-condition formula could not be parsed. Similar parsing issues are observed in `xterm`, where a high number of unclosed annotations and line-continuation errors occur despite the repository's relatively small size.

These errors indicate that conditional compilation in real-world systems is often irregular and shaped by build-system constraints rather than strictly adhering to idealised preprocessor usage. Qualitative inspection shows that such cases typically arise from complex commented-out directives, or unconventional formatting. As a result, not all commits can be analysed reliably, and parsing errors represent both a limitation of the analysis and empirical evidence of variability-related complexity.

Quantitative Results The analysed subject systems differ strongly with respect to error-related changes. In `libssh`, a substantial proportion of the analysed commits are classified as error-related based on commit-message keywords. This indicates that corrective maintenance is a frequent activity in the evolution of variability-related code. Many of these commits are asso-

```
==== Topic Analysis: Refactoring / Reconfiguration / AddWithMapping ====

Number of matching commits: 9
Number of messages loaded : 9

Keywords ordered by frequency:
fix                  3
support              3
add                  2
missing              2
ifdef                2
ssh                  2
agent                2
build                1
windows              1
ifdefs               1
cleanup              1
define               1
check                1
libssh               1
header               1
poll                 1
channel_poll         1
not                  1
yet                  1
channel_select       1
win32                1
bug                  1
debian               1
rules                1
executable           1
experimental         1

...
```

Figure 4.3: Keyword frequency results for topic analysis of commits classified as Refactoring, Reconfiguration, or AddWithMapping in libssh.

ciated with refactoring or reconfiguration edits, suggesting that higher-level changes to presence conditions are often motivated by the need to correct incorrect or incomplete configurations rather than by feature evolution alone.

In contrast, `xterm` exhibits no commits that are classified as error-related based on commit-message analysis, despite the presence of numerous parsing and annotation-related issues. This discrepancy highlights that the absence of explicit error-related keywords in commit messages does not imply the absence of underlying problems. Instead, it reflects differences in developer communication practices and conventions regarding how issues are documented.

Qualitative Inspection To complement the quantitative analysis, selected error-related commits were inspected manually. In most cases, the commit message accurately reflects the nature of the change, and the corresponding variability-aware diff confirms that the commit addresses an actual defect. Typical examples include correcting incorrect `#ifdef` conditions, adding missing error checks under specific configurations, or fixing platform-dependent compilation paths.

A recurring pattern is that many error-related commits address misconfigurations introduced by earlier changes. Edits to conditional compilation directives may appear local but can affect a

large portion of the configuration space. As a result, developers often discover unintended effects only after the fact, leading to subsequent corrective commits. This explains why similar corrective patterns recur over time.

At the same time, some commits flagged as error-related primarily address maintainability concerns, such as build warnings or code cleanup, rather than functional defects. These cases illustrate that commit-message-based classification tends to over-approximate error-related changes and should be interpreted as an indicator of corrective intent rather than a precise fault classification.

Overall, the results show that error correction is a central driver of variability evolution in actively maintained systems such as `libssh`. By combining variability-aware structural analysis with commit-message inspection, the evaluation captures both explicit and implicit error-related activities.

4.2.5 Threats to Validity

[4]

The evaluation is subject to several threats to validity. External validity is limited by the selection of subject systems. Only two repositories were analysed, and both are implemented in C using preprocessor-based variability. While these systems are representative of a common class of configurable software, the results may not generalise to other languages or variability mechanisms.

Scalability also poses a limitation. For large repositories with extensive commit histories, repeated initialisation and output overwriting during multi-threaded execution reduced the number of commits available for quantitative analysis. As a result, some findings are interpreted qualitatively rather than as absolute frequencies. Analysing larger numbers of systems and improving result aggregation are considered out of scope for this thesis.

Internal validity is affected by the use of keyword-based heuristics for identifying error-related commits. While manual inspection suggests that many classifications are accurate, the heuristic may both miss error-related commits and incorrectly flag maintenance-oriented changes as bug-related. This threat is mitigated by combining commit-message analysis with variability-aware structural classification.

4.2.6 Discussion

The evaluation demonstrates that variability-related changes are closely tied to error correction and maintenance activities. In particular, the results show that developer mistakes in conditional compilation are not necessarily frequent in terms of explicit documentation, but they can be severe in impact, affecting large parts of the configuration space and requiring subsequent corrective commits.

The combination of variability-aware differencing and commit-message analysis provides a more transparent view of how presence conditions evolve over time. While commit messages alone cannot fully explain variability evolution, they provide valuable contextual information when interpreted alongside structural edit classifications.

Future work could extend this approach to additional subject systems, improve scalability of result collection, or incorporate more advanced NLP techniques. Moreover, integrating variability-aware analysis directly into development tools could help developers better anticipate the semantic

impact of changes to conditional compilation and reduce the likelihood of configuration-dependent errors.

Chapter 5 Conclusion

This thesis investigated changes in variability in configurable software systems, with a particular focus on presence conditions implemented, using the C/C++ preprocesser. The motivation for this work was that traditional text-based diff tools are limited in their ability to reveal the semantic impact of such changes. Even small modifications to conditional compilation expressions can significantly alter which configurations include specific code, without this being clearly visible in conventional version control diffs.

To address this challenge DiffDetective was used as a variability-aware differencing framework, that classifies changes according to their semantic effect on variability, including Refactoring, Reconfiguration, Generalization and Specialization. Building on this framework a custom analysis component was implemented to extract commit data, store classification results and associate variability changes with commit messages. This enabled an investigation of both the technical nature of variability changes, and how these changes are communicated by developers.

The results show that variability-related changes are a recurring part of real-world software, and are often introduced during maintenance, build fixes or platform-related adjustments. While some commit messages explicitly reference issues with conditional compilation, many variability-changing commits do not clearly indicate their potential impact. This suggests that changes affecting variability are not always treated as conceptually distinct from other code modifications, despite their possible consequences for the configuration space.

5.1 Git Repository

https://github.com/sakha22sdu/Bachelor_Project1/tree/main.

List of Figures

2.1 Classification of edits to configurable software proposed by Bittner et al. (2022). . 6

4.1 Comparison of commit-message styles in xterm and libssh. xterm commits use short, identifier-based messages with little semantic information, whereas libssh commits explicitly document corrective intent and error-related changes. 17

4.2 Example of a bug-related reconfiguration commit in `libssh`. The commit message explicitly indicates corrective intent, and the edit class shows that an existing presence condition was modified rather than new functionality being added. 17

4.3 Keyword frequency results for topic analysis of commits classified as Refactoring, Reconfiguration, or AddWithMapping in libssh. 19

List of Tables

Bibliography

[1] Sven Apel et al. *Feature-Oriented Software Product Lines: Concepts and Implementation*. Springer, 2013. ISBN: 978-3-642-37520-0. DOI: 10.1007/978-3-642-37521-7.

[2] Peter Bittner et al. "A Classification of Variability Edits in Source Code". In: *IEEE Transactions on Software Engineering* 48.11 (2022), pp. 4420–4441. DOI: 10.1109/TSE.2021.3126330.

[3] Jörg Liebig et al. "An Analysis of the Variability in Forty Preprocessor-Based Software Product Lines". In: *Proceedings of the 32nd International Conference on Software Engineering (ICSE)*. ACM, 2010, pp. 105–114. DOI: 10.1145/1806799.1806815.

[4] Claes Wohlin et al. *Experimentation in Software Engineering, Second Edition*. Springer, 2024. DOI: 10.1007/978-3-662-69306-3.